S02 Singleton Managers

CRE132 worksheet

What you'll make

A **GameManager**: one object that keeps the score and lasts for the whole game. Coins add to the score when the player touches them, and a debug key reloads the level. The coins come back, but the score carries on.

By the end you'll know what a **singleton** is (one of something, reachable from anywhere), what `static` means, and how `DontDestroyOnLoad` keeps an object alive when a new scene loads.

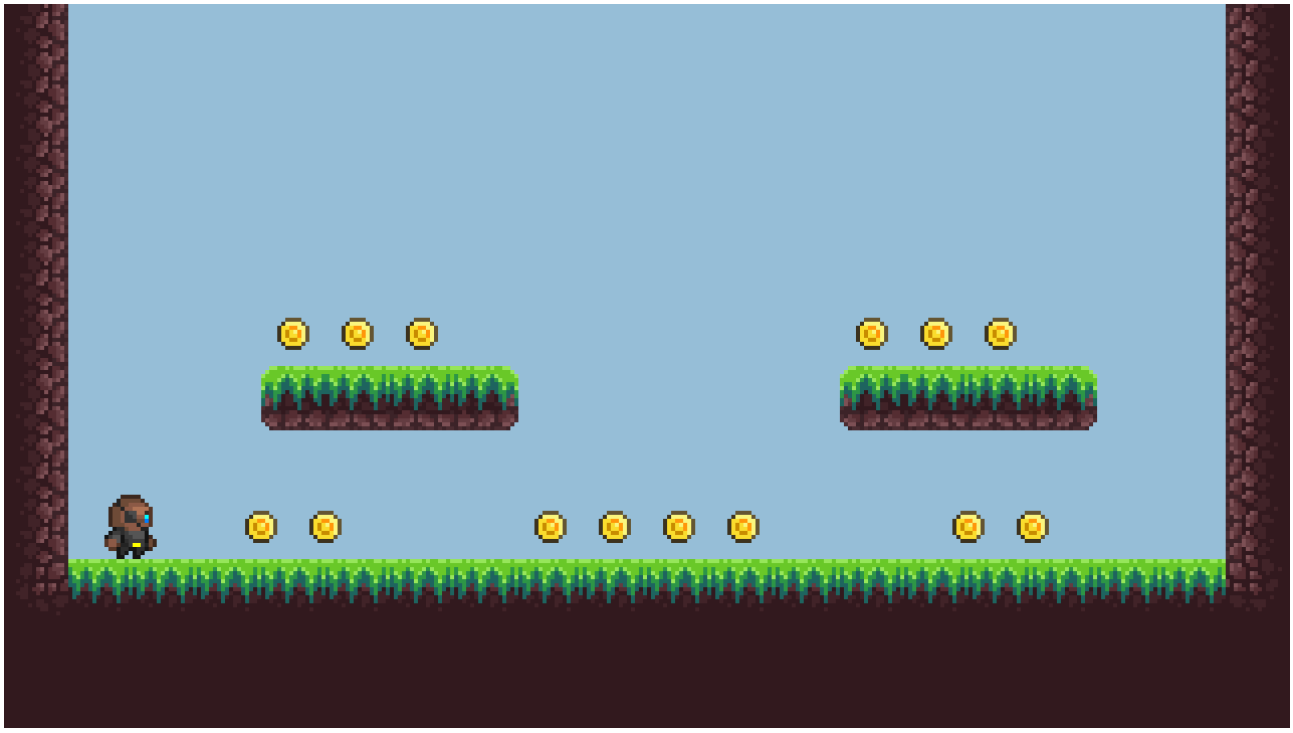


Figure 1: The finished scene: the player in a room with coins on the ground and on two platforms.

Before you start

1. Unzip `CRE132_S02_SingletonManagers_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S02_SingletonManagers_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S02_SingletonManagers`.
3. Press **Play**. You can walk (A/D or arrows), run (hold Shift) and jump (Space), and the coins spin, but walking into them does nothing yet. Press Play again to stop.

The scene has the **Player**, fourteen **Coins** (inside **Coins**), and two empty objects called **GameManager** and **Scene Reloader**. You'll give them their scripts.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The GameManager

A coin needs to add to the score, but the coin is destroyed when it's collected, and the score has to outlive the coin and later the whole level. You could drag a score object into every coin's Inspector, but that's tedious and doesn't work for coins made while the game runs. A singleton solves this.

Make a folder `Assets/_game/Scripts/Managers` and create a script in it called `GameManager`:

```
// The GameManager: one object that lasts for the whole game and keeps the score.
// Any script can reach it with GameManager.Instance, without a reference set in
// the Inspector.
using UnityEngine;

public class GameManager : MonoBehaviour
{
    // The one GameManager. Static means it belongs to the class, not to any one
    // object, so GameManager.Instance works from anywhere. Only this script can
    // change it (private set).
    public static GameManager Instance { get; private set; }

    // Serialized so you can watch it change in the Inspector while playing.
    [SerializeField] private int score;

    public int Score
    {
        get { return score; }
    }

    private void Awake()
    {
        // A GameManager already exists (we came back to a scene that has one in
        // it), so this copy isn't needed.
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;

        // Keep this object when a new scene loads, so the score carries on.
        DontDestroyOnLoad(gameObject);
    }

    public void AddScore(int amount)
    {
        score = score + amount;
        Debug.Log("Score: " + score);
    }
}
```

What's going on:

- **`static`** means `Instance` belongs to the `GameManager` class itself, not to one object. That's why any script can write `GameManager.Instance` without a reference.
- **`{ get; private set; }`** lets everyone read `Instance`, but only the GameManager can set it. It does that in `Awake`: `Instance = this;`.
- **`DontDestroyOnLoad`** moves the object out of the scene, so it isn't destroyed when a scene loads. It only works on objects at the top of the Hierarchy, not on children.

- **The `if` at the top of `Awake`** deals with copies. Each time the scene loads, its own GameManager wakes up. If one already exists, the new one destroys itself, so there is only ever one.

Add it: select the **GameManager** object and click **Add Component > Game Manager**. Leave **Score** at 0.

2. Coins that score: ScoreCoin

Make a folder `Assets/_game/Scripts/S02_SingletonManagers` and create `ScoreCoin` in it:

```
// A coin that adds to the GameManager's score when the player touches it.
using UnityEngine;

public class ScoreCoin : MonoBehaviour
{
    [SerializeField] private int value = 1;

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            // No Inspector reference needed: the singleton is reachable from anywhere.
            GameManager.Instance.AddScore(value);
            Destroy(gameObject);
        }
    }
}
```

- **`CompareTag("Player")`** checks the tag on the thing that touched the coin. The Player object is tagged **Player**, so only the player collects coins.
- **`GameManager.Instance.AddScore(value)`** is the singleton at work: the coin has no slot to fill, it just asks the class for its one GameManager.

Add it: expand **Coins** in the Hierarchy, select all fourteen **Coin** objects (click the first, Shift-click the last) and add **Score Coin**. Leave **Value** at 1. Their colliders already have **Is Trigger** ticked.

Test it: press Play and collect some coins. The Console shows `Score: 1`, `Score: 2`… Select the **GameManager** while playing and watch its **Score** field go up. Look at the Hierarchy too: the GameManager now sits under a separate heading called **DontDestroyOnLoad**.

3. Reloading the level: SceneReloader

Create `SceneReloader` in the `S02_SingletonManagers` folder:

```
// Press R to reload the current scene. The coins come back, but the GameManager
// (and the score) survive.
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;

public class SceneReloader : MonoBehaviour
{
    private void Update()
    {
        // A debug key, so we read the keyboard directly instead of making an action
        // for it.
        if (Keyboard.current != null && Keyboard.current.rKey.wasPressedThisFrame)
        {
            string sceneName = SceneManager.GetActiveScene().name;
```

```
            SceneManager.LoadScene(sceneName);
        }
    }
}
```

- **`SceneManager.GetActiveScene().name`** is the name of the scene you're in, so `LoadScene` loads it again from the start.
- **`Keyboard.current.rKey`** reads the R key directly. That's fine for a debug key like this; keys for actual gameplay still go through input actions.
- **`LoadScene` only finds scenes in the build list** (**File > Build Profiles**, **Scene List**). This scene is already in it.

Add it to the **Scene Reloader** object.

Test it: press Play, collect a few coins, then press **R**. The coins come back and the next coin carries on from your old score.

4. Why the two key lines matter

Two quick experiments. Undo each one before moving on.

1. **Without DontDestroyOnLoad:** put `//` in front of the `DontDestroyOnLoad(gameObject);` line, save, and play. Collect coins and press R. The score goes back to 0, because the GameManager was destroyed with the old scene and the new scene's GameManager starts again.
2. **Without the duplicate check:** remove the `//` again, then comment out the whole `if (Instance != null && Instance != this)` block (the `if` line, both braces and the two lines inside). Play and press R a few times. Look under **DontDestroyOnLoad** in the Hierarchy: there's a new GameManager for every reload. Put the block back and the newcomer destroys itself again.

Check your work

- ☐ Each coin adds 1 to the score in the Console and on the GameManager's **Score** field.
- ☐ Pressing R brings the coins back, and the score carries on from where it was.
- ☐ During Play, there's one GameManager under **DontDestroyOnLoad**, however many times you reload.
- ☐ Compare with `CRE132_S02_SingletonManagers_Finished` if something behaves differently.

Key points

- **A singleton is "one of these, reachable from anywhere".** It's good for managers (game, audio, scenes), not for everything: the player and enemies should stay normal objects.
- **`static`** belongs to the class, not to an object. `GameManager.Instance` works from any script without an Inspector slot.
- **`DontDestroyOnLoad`** keeps an object alive across scene loads. It only works on root objects (not children), and you need the duplicate check so copies don't pile up.

If something goes wrong

- **NullReferenceException in ScoreCoin when you touch a coin:** `GameManager.Instance` is `null`. Check that the **GameManager** object has the GameManager component and that it's enabled.
- **Coins do nothing:** check that the Player is tagged **Player** (top of the Inspector), and that the Coin colliders have **Is Trigger** ticked.
- **"Scene … couldn't be loaded because it has not been added to the build settings":** add the scene in **File > Build Profiles**, **Scene List** (**Add Open Scenes**).
- **The score resets on R:** the `DontDestroyOnLoad(gameObject);` line is missing or still commented out.

Try it

1. Add a `ResetScore()` method to the GameManager that sets the score back to 0, and call it when the player presses Backspace. (Hint: copy the way SceneReloader reads the R key; Backspace is `Keyboard.current.backspaceKey`.)
2. Make the coins on the platforms worth 5, using the Inspector only.
3. Add a `lives` field to the GameManager and check that it lasts across reloads too.